Birds AI Sound Classifier - Prozess- & Überwachungskonzept (Version V1.2)

Dokumentation: Zusammenspiel von 'autostart_monitor.pyw' ('autostart_monitor.py') und 'birds_ai_sound_classifier.py'
Datum: 11. November 2026
Status: Freigegeben

---

1. Übersicht & Rollenverteilung

'autostart_monitor.pyw' (bzw. 'autostart_monitor.py', Version V1.2) und 'birds_ai_sound_classifier.py' bilden ein aufeinander abgestimmtes Gespann zur zuverlässigen Ausführung, Selbstheilung und Überwachung der Bioakustik-Anwendung ohne externe Batch-Dateien ('start.bat' und 'autostart_monitor.exe' sind entfallen und werden bei Updates automatisch bereinigt).

* 'autostart_monitor.pyw' / 'autostart_monitor.py' (Watchdog / Supervisor, V1.2):
Wird fensterlos über 'pythonw.exe' (ohne eigenes schwarzes CMD-Fenster im Hintergrund) ausgeführt, stellt eine kompakte Tkinter-GUI bereit und übernimmt die kontinuierliche Prozessüberwachung im 60-Sekunden-Takt, die Erkennung und Bereinigung doppelter Instanzen, das Aufräumen verwaister Hintergrundprozesse, die automatische Migration/Bereinigung einer alten 'autostart_monitor.exe' sowie den direkten Neustart der Hauptanwendung über 'python.exe'.
* 'birds_ai_sound_classifier.py' (Hauptanwendung):
Führt die Webserver-Dienste (Flask / Waitress), die mDNS-Bekanntmachung, den nächtlichen Auto-Update-Scheduler, die Community-Versionsmeldung (CounterAPI.dev) sowie den Audio-Erkennungsdienst aus (ausgelagert in einen separaten Multiprocessing-Kindprozess). Zudem überwacht und aktualisiert die Hauptanwendung bei Updates automatisch den Watchdog 'autostart_monitor.pyw'.

Komponente	Rolle	Hauptaufgabe
'autostart_monitor.pyw' / '.py'	Supervisor / Wächter (V1.2)	Statusprüfung alle 60s, Migration alter '.exe', Killen von Duplikaten/Zombies, Direktstart via 'python.exe'
'birds_ai_sound_classifier.py'	Hauptanwendung	Flask/Waitress-Webserver, mDNS, Auto-Update, CounterAPI.dev & Audioanalyse-Multiprocessing

---

2. Startprozedur & Ausführungskontext ('start_app' & 'get_python_exe')

Da 'autostart_monitor.pyw' über 'pythonw.exe' (GUI-Subsystem ohne Konsole) läuft, die Hauptanwendung 'birds_ai_sound_classifier.py' jedoch ein sichtbares Konsolenfenster für Status- und Logausgaben benötigt, ist der Startvorgang speziell darauf abgestimmt:

1. Zuverlässige Ermittlung von 'python.exe' ('get_python_exe'):
Unabhängig davon, ob der Monitor über 'pythonw.exe' oder 'pyw.exe' gestartet wurde, ermittelt 'get_python_exe()' automatisch den zugehörigen Konsolen-Interpreter 'python.exe' im selben Python-Installationsverzeichnis.
2. Direkter Start ohne 'start.bat':
'start_app()' startet 'birds_ai_sound_classifier.py' direkt über '[PYTHON_EXE, target_path]' im Anwendungsverzeichnis ('cwd=APP_DIR') mit dem Flag 'subprocess.CREATE_NEW_CONSOLE'. Die frühere Wrapper-Datei 'start.bat' wird nicht mehr benötigt (und bei Updates automatisch entfernt), wodurch auch keine blockierenden Batch-Abfragen ('"Batchvorgang abbrechen (J/N)?"') mehr auftreten können.
3. Garantiert sichtbares Konsolenfenster ('SW_SHOWNORMAL'):
Über 'startupinfo.dwFlags |= subprocess.STARTF_USESHOWWINDOW' und 'startupinfo.wShowWindow = 1' ('SW_SHOWNORMAL') sowie den bewussten Verzicht auf 'stdin=DEVNULL' (da 'pythonw.exe' sonst 'STARTF_USESTDHANDLES' mit ungültigen Standard-Handles an das neue Konsolenfenster vererben würde) wird sichergestellt, dass das Konsolenfenster der Hauptanwendung unter Windows jederzeit sichtbar und fehlerfrei geöffnet

wird.

---

3. Automatische Migration & Bereinigung alter Monitor-Versionen ('cleanup_legacy_exe_monito

Um bestehende Installationen nahtlos von der früheren kompilierten 'autostart_monitor.exe' auf die direkt aktualisierbare 'autostart_monitor.pyw' umzustellen, greifen zwei Mechanismen ineinander:

1. Beim Start und nach Updates in 'birds_ai_sound_classifier.py' ('_start_or_restart_monitor_pyw'):
* Fehlen 'autostart_monitor.pyw' oder 'autostart_monitor.py' lokal, werden sie automatisch von GitHub nachgeladen.
* Wird noch eine laufende oder vorhandene 'autostart_monitor.exe' erkannt (oder wurde gerade ein Software-Update eingespielt), startet die Hauptanwendung 'autostart_monitor.pyw' automatisch über 'pythonw.exe' neu.
2. In 'autostart_monitor.pyw' ('cleanup_legacy_exe_monitor'):
* Beendet beim Start überzählige ältere '.py'/'.pyw'-Monitor-Prozesse.
* Erkennt laufende Prozesse der alten 'autostart_monitor.exe', beendet diese sauber ('terminate()' / 'kill()') und löscht anschließend die Datei 'autostart_monitor.exe' dauerhaft aus dem Installationsordner.

---

4. Erkennung doppelter Instanzen (Duplicate Detection)

Die Erkennung laufender Instanzen erfolgt über 'get_running_instances()' und wird periodisch im 'monitor_loop()' ausgeführt:

* Filterkriterien:
Alle Systemprozesse werden mit 'psutil' nach Prozessen durchsucht, deren Name 'python' enthält und in deren Befehlszeile ('cmdline') der Name 'birds_ai_sound_classifier.py' vorkommt (unter Ausschluss von 'multiprocessing.spawn').
* Keine Fehlalarme durch Multiprocessing-Kinder:
Unter Windows erzeugt Pythons 'multiprocessing'-Modul den Audio-Kindprozess mit dem Argument 'multiprocessing.spawn'. Dieser wird explizit herausgefiltert und daher nicht als zweite Hauptinstanz gewertet.
* Priorisierung nach Alter (FIFO-Prinzip):
Werden mehrere Instanzen festgestellt ('len(instances) > 1'), sortiert der Monitor die Liste chronologisch nach 'create_time':
* Die älteste Instanz ('instances[0]') bleibt unberührt im laufenden Betrieb.
* Alle überzähligen, neueren Instanzen ('instances[1:]') werden mitsamt ihrem Prozessbaum über 'kill_process_tree()' beendet.

---

5. Prozess-Terminierung & Waisen-Bereinigung

* 'kill_process_tree(proc)' (5-stufige Kaskade):
1. Ermittelt eventuelle übergeordnete 'cmd.exe'-Elternprozesse über 'proc.parents()' und die 'ppid'-Kette.
2. Erfasst alle untergeordneten Kindprozesse ('proc.children(recursive=True)', z. B. den Audioanalyse-Subprozess).
3. Sendet zunächst 'terminate()' an alle Kindprozesse und den Hauptprozess.
4. Wartet über 'psutil.wait_procs(..., timeout=3)' bis zu 3 Sekunden und beendet nicht reagierende Prozesse hart mit 'kill()'.
5. Schließt abschließend eventuell verbliebene 'cmd.exe'-Elternfenster.
* 'cleanup_orphaned_processes()':
Ist die Hauptanwendung beendet oder abgestürzt ('len(instances) == 0'), bereinigt der Monitor vor dem Neustart alle im

Anwendungsverzeichnis verbliebenen 'multiprocessing.spawn'-Worker sowie eventuelle verwaiste 'cmd.exe'-Fenster (während er sich selbst über '"autostart_monitor" in cmd_str' schützt). Dadurch werden blockierte Mikrofon-Streams oder gesperrte SQLite-Datenbanken zuverlässig vermieden.

---

6. Wichtige Betriebs- und Praxishinweise

* Prüfintervall (60 Sekunden):
Die Überwachungsschleife prüft im 60-Sekunden-Takt (in 1-Sekunden-Schritten unterteilt, damit sich das Monitor-Fenster beim Schließen sofort beendet).
* Empfohlene Autostart-Einbindung:
In den Windows-Autostart ('shell:startup') wird eine Verknüpfung auf 'autostart_monitor.pyw' gelegt. Der Monitor startet fensterlos im eigenen GUI-Fenster und übernimmt den Start von 'birds_ai_sound_classifier.py' vollautomatisch.
* Zusammenspiel mit GitHub Software-Updates (Manuell & Auto-Update 02:00 Uhr):
Wird über 'settings.html' ein manuelles Software-Update ausgelöst oder um 02:00 Uhr nachts vollautomatisch ein neues GitHub-Release installiert, legt 'birds_ai_sound_classifier.py' zunächst automatisch ein Datenbank-Backup ('backup_db/') an, aktualisiert auch 'autostart_monitor.py' und 'autostart_monitor.pyw', startet den neuen 'autostart_monitor.pyw' über 'pythonw.exe' frisch und beendet sich anschließend selbst. 'autostart_monitor.pyw' startet die aktualisierte Hauptanwendung daraufhin automatisch neu, woraufhin die neue Version beim Start über 'report_version_update_thread()' an CounterAPI.dev gemeldet wird.